

DESIGN AND SIMULATION OF AN 8-BIT ARITHMETIC LOGIC UNIT (ALU) USING VERILOG HDL

Submitted By: Kashika Sharma

Software Used: Xilinx Vivado

Branch: EE-VDT (Electronics Engineering – VLSI Design and Technology)

ABSTRACT

An Arithmetic Logic Unit (ALU) is one of the most important components of a processor. It performs arithmetic and logical operations required by digital systems. In this project, an 8-bit ALU was designed using Verilog Hardware Description Language (HDL) and implemented using Xilinx Vivado. The ALU supports eight different operations controlled by a 3-bit opcode. Functional verification was performed using a Verilog testbench and simulation waveforms, followed by synthesis to generate the hardware implementation. The project demonstrates the principles of digital design and hardware description languages.

INTRODUCTION

An Arithmetic Logic Unit (ALU) is a combinational digital circuit used to perform arithmetic and logical operations on binary numbers. It forms the core computational unit of microprocessors, microcontrollers, and digital systems.

The ALU receives two operands and an operation selection signal. Based on the control input, it performs the required operation and produces the corresponding output. Arithmetic operations include addition and subtraction, whereas logical operations include AND, OR, XOR, and NOT. Shift operations are also commonly implemented.

This project focuses on designing an 8-bit ALU using Verilog HDL and verifying its functionality through simulation in Xilinx Vivado.

OBJECTIVES

The objectives of this project are:

- To design an 8-bit Arithmetic Logic Unit using Verilog HDL.
- To perform arithmetic and logical operations.
- To verify the design using a testbench.
- To simulate the functionality using Xilinx Vivado.
- To synthesize the design and generate RTL and schematic diagrams.
- To understand the fundamentals of digital system design.

SOFTWARE AND TOOLS USED

Hardware Description Language: Verilog HDL

Software: Xilinx Vivado

Simulation Tool: XSIM Simulator

Design Type: Register Transfer Level (RTL)

SYSTEM SPECIFICATIONS

Inputs

Signal	Width	Description
A	8-bit	First Operand
B	8-bit	Second Operand
Opcode	3-bit	Operation Selection

Outputs

Signal	Width	Description
Result	8-bit	Output of ALU
Zero	1-bit	Indicates Result = 0

OPERATIONS PERFORMED

The ALU supports eight operations selected by a 3-bit opcode.

Opcode	Operation
000	Addition
001	Subtraction
010	AND
011	OR
100	XOR
101	Left Shift
110	Right Shift
111	NOT

DESIGN METHODOLOGY

The design was implemented using Verilog HDL. Two 8-bit inputs A and B and a 3-bit opcode are provided to the ALU.

Depending on the opcode, the ALU performs one of eight operations. The result is stored in an 8-bit output signal. A zero flag is generated whenever the output becomes zero.

The design was verified using a separate testbench which applied different combinations of inputs and operation codes.

VERILOG IMPLEMENTATION

The ALU module consists of:

- Input A[7:0]
- Input B[7:0]
- Input Opcode[2:0]
- Output Result[7:0]
- Output Zero

The functionality is implemented using a case statement inside an always block. Each opcode corresponds to a specific operation.

```
1  `timescale 1ns / 1ps
2  module alu_8bits(
3      input [7:0] A,
4      input [7:0] B,
5      input [2:0] Opcode,
6      output reg[7:0] result,
7      output reg zero
8  );
9  always @(*)
10 begin
11
12 case (Opcode)
13
14 3'b000: result = A + B;
15
16 3'b001: result = A - B;
17
18 3'b010: result = A & B;
19
20 3'b011: result = A | B;
21
22 3'b100: result = A ^ B;
23
24 3'b101: result = A << 1;
25
26 3'b110: result = A >> 1;
27
28 3'b111: result = ~A;
29
30 endcase
31
32 if(result == 8'b00000000)
33     zero = 1;
34 else
35     zero = 0;
36
37 end
38
39 endmodule
```

TESTBENCH DESIGN

A Verilog testbench was created to verify the operation of the ALU. Various input combinations were applied for different opcodes to test arithmetic and logical functions. The following operations were verified:

- Addition
- Subtraction
- AND
- OR
- XOR
- Shift Left
- Shift Right
- NOT

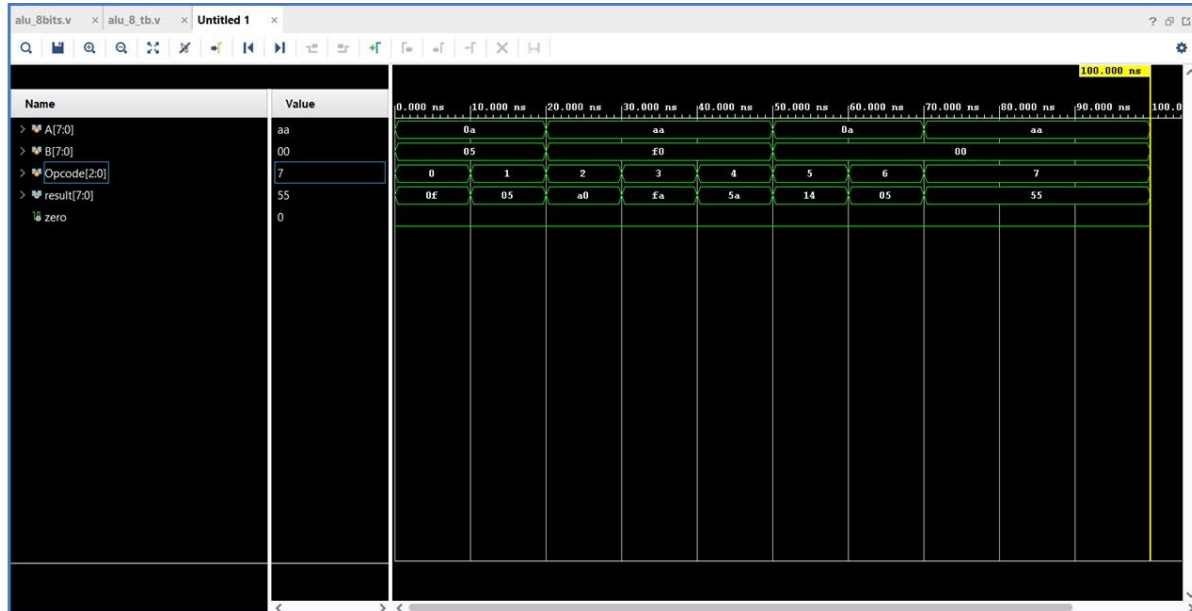
Simulation delays were introduced between each operation to observe output changes clearly.

```
1  `timescale 1ns / 1ps
2  module alu_8_tb;
3
4  reg [7:0] A;
5  reg [7:0] B;
6  reg [2:0] Opcode;
7
8  wire [7:0] result;
9  wire zero;
10
11  alu_8bits uut(
12      .A(A),
13      .B(B),
14      .Opcode(Opcode),
15      .result(result),
16      .zero(zero)
17  );
18
19  initial
20  begin
21
22      // Addition
23      A = 8'd10;
24      B = 8'd5;
25      Opcode = 3'b000;
26      #10;
27
28      // Subtraction
29      A = 8'd10;
30      B = 8'd5;
31      Opcode = 3'b001;
32      #10;
33
34      // AND
35      A = 8'b10101010;
36      B = 8'b11110000;
37      Opcode = 3'b010;
38      #10;
39
40      // OR
41      A = 8'b10101010;
42      B = 8'b11110000;
43      Opcode = 3'b011;
44      #10;
45
46      // XOR
47      A = 8'b10101010;
48      B = 8'b11110000;
49      Opcode = 3'b100;
50      #10;
51
52      // Shift Left
53      A = 8'd10;
54      B = 8'd0;
55      Opcode = 3'b101;
56      #10;
57
58      // Shift Right
59      A = 8'd10;
60      B = 8'd0;
61      Opcode = 3'b110;
62      #10;
63
64      // NOT
65      A = 8'b10101010;
66      B = 8'd0;
67      Opcode = 3'b111;
68      #10;
69
70  end
71
72  endmodule
```

SIMULATION RESULTS

Functional simulation was carried out using the XSIM simulator available in Xilinx Vivado. The simulation waveform confirmed the proper execution of all operations. The output values changed correctly according to the applied opcode and input values. The zero flag was also verified successfully.

Simulation Waveform



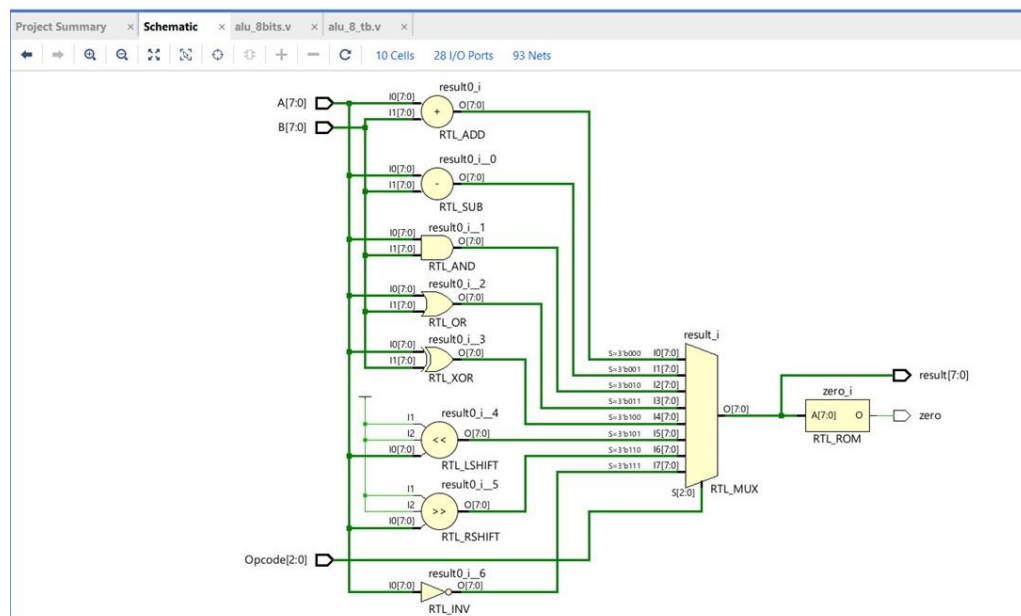
RTL SCHEMATIC

After successful simulation, RTL synthesis was performed. The RTL schematic represents the logical structure of the ALU before hardware mapping.

The RTL view shows:

- Inputs A and B
- Opcode lines
- Multiplexing logic
- Arithmetic and logical blocks
- Output result and zero flag

RTL Schematic



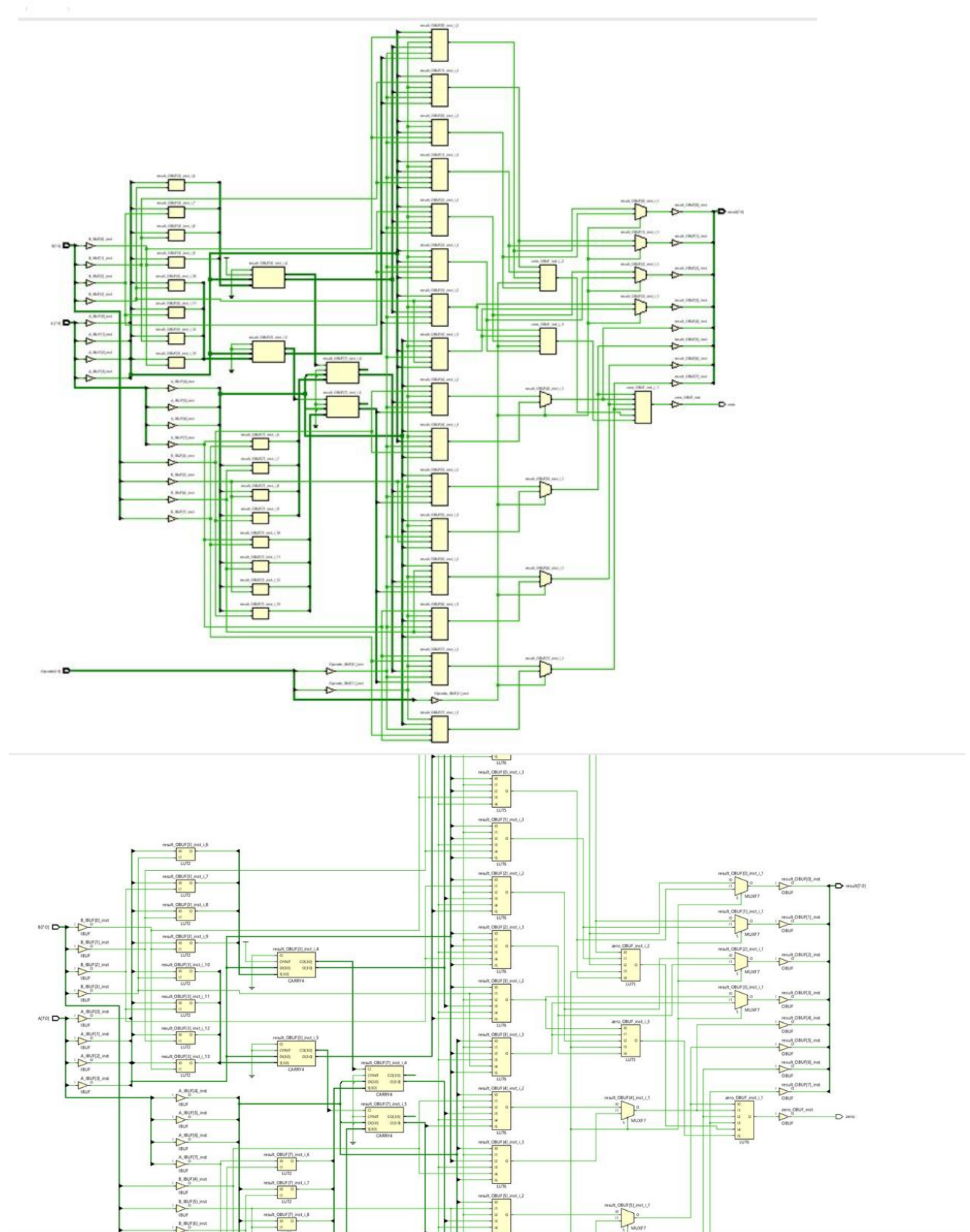
SYNTHESIS RESULTS

The design was synthesized successfully using Xilinx Vivado.

The synthesis process generated the hardware representation of the ALU and provided resource utilization information.

The synthesized schematic confirms that the Verilog code is suitable for FPGA implementation.

Synthesis Schematic



CHAPTER 12: APPLICATIONS

The Arithmetic Logic Unit is widely used in:

- Microprocessors
- Microcontrollers
- Digital Signal Processors (DSP)
- Embedded Systems
- Computer Architecture
- Arithmetic Circuits
- FPGA-based Digital Systems

CHAPTER 13: ADVANTAGES

- High-speed operation.
- Efficient implementation using Verilog HDL.
- Easy hardware synthesis.
- Supports multiple arithmetic and logical functions.
- Suitable for FPGA implementation.
- Modular and scalable design.

CHAPTER 14: CONCLUSION

In this project, an 8-bit Arithmetic Logic Unit was successfully designed and implemented using Verilog HDL. Functional verification was carried out using a Verilog testbench and simulation in Xilinx Vivado. The design supported eight arithmetic and logical operations controlled by a 3-bit opcode. The synthesis results and RTL schematic confirmed the correctness of the design. This project provides a strong foundation for understanding digital system design and FPGA-based hardware implementation.

REFERENCES

1. M. Morris Mano, *Digital Design*, Pearson Education.
2. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Pearson.
3. Xilinx Vivado Design Suite User Guide.
4. IEEE Standard for Verilog Hardware Description Language.